HOA SEN UNIVERSITY
RESPECT DIVERSITY

Đệ quy
(Recursion)

Cấu trúc dữ liệu & Giải thuật – SW102DV01

(Data Structures & Algorithms)

Vũ Đình Khôi

Nội dung

- Khái niệm, bài toán minh họa
- Xây dựng giải thuật đệ quy
- Phân loại đệ quy
- Ưu điểm & khuyết điểm của giải thuật đệ quy
- Đệ quy & bộ nhớ stack
- Khử đệ quy

Nội dung

- Một số bài toán điển hình
  - Bài toán lũy thừa (Exponentiation)
  - Bài toán tháp Hà Nội (Tower of Hanoi)
  - Bài toán thỏ sinh sản (Rabbit problem)
  - Phát sinh các số nhị phân (Generating binary numbers)
  - Nghịch đảo các phần tử trong danh sách (Reversed list),
  - Nghịch đảo các chữ số (Reversed numbers)

Tham khảo

- Chapter 5. Robert Sedgewick. (1998). *Algorithms in C, Part 1-4, Third Edition*. Addison-Wesley
- Chapter 5. Robert Sedgewick. (2003). *Algorithms in Java, Part 1-4, Third Edition*. Addison-Wesley

Giới thiệu

I find that I don't understand things unless I try to program them.

Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it

-- Richard Feynman

Khái niệm đệ quy (Recursion)

- Đệ quy – recursion (递归)
  - Đệ (递): đưa, chuyển giao, đệ trình
  - Quy (归): quay về
  - Recur: trở lại vấn đề
- Quay về chuyển giao kết quả sau khi tính toán
- Chuyển việc tính toán (giải quyết vấn đề) bằng cách quay trở lại tính toán các vấn đề ở cấp độ nhỏ hơn hoặc giải quyết các trường hợp cơ sở (base case/basis case).
- [Toán học] việc gọi một hàm (function) chính nó và chỉ ra trường hợp cơ sở (base case).
  - Cách tính lũy thừa (exponentiation)
  - Cách tính giai thừa (factorial)

Khái niệm đệ quy (Recursion)

Cho S(n) = 1 + 2 + 3 + … + n

=>S(10)? S(11)?

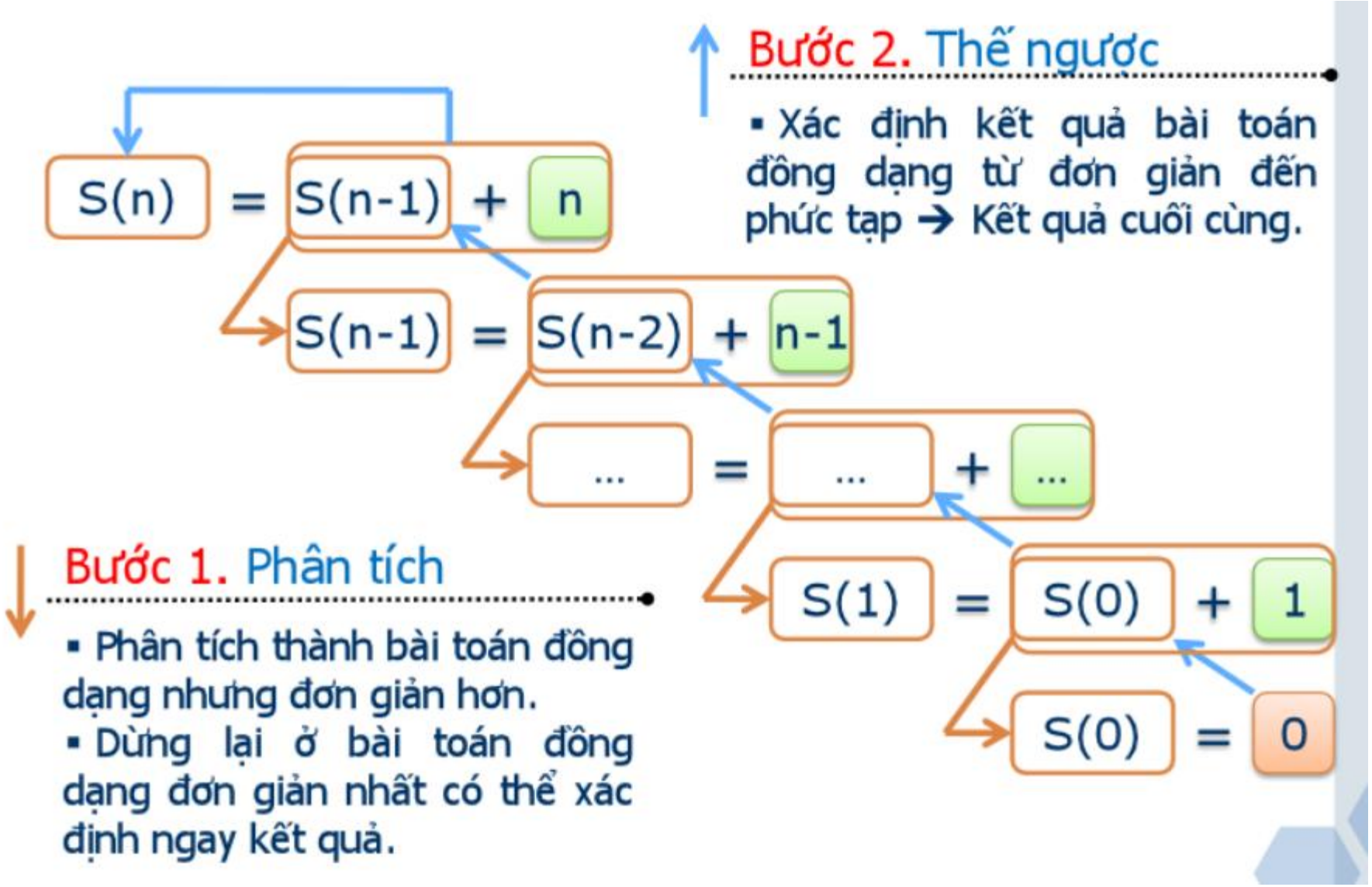
$$S(10) = 1 + 2 + \ldots + 10 = 55$$

$$S(11) = \boxed{1 + 2 + \ldots + 10} + 11 = 66$$

$$= S(10) + 11$$

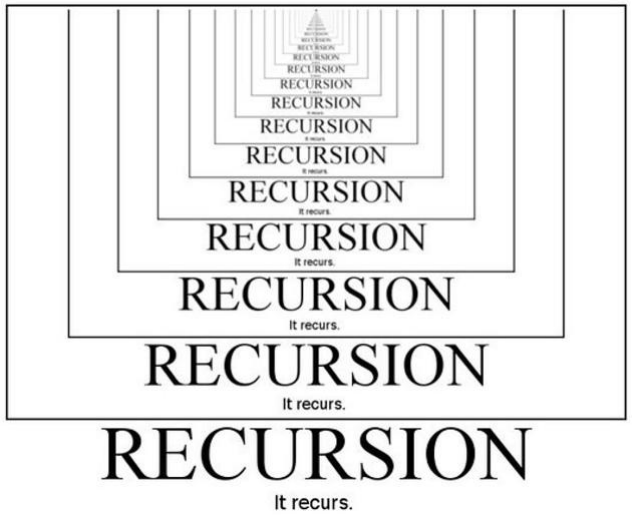
$$= 55 + 11 = 66$$

Khái niệm đệ quy (Recursion)



Khái niệm đệ quy (Recursion)

- Một chương trình/hàm đệ quy gọi lại chính nó (calls itself)
  - Tuy nhiên, nó không thể gọi mãi mãi → phải có điều kiện dừng (termination condition) hay trường hợp cơ sở (base case).
- Thuật toán đệ quy (recursive algorithm)
  - Giải quyết vấn đề bằng cách giải một hoặc nhiều trường hợp nhỏ hơn của cùng một vấn đề.
- Sử dụng đệ quy để giải một số bài toán một cách tự nhiên và đẹp (natural and elegant solution)



Bài toán nhân 2 số nguyên: a * b

```
// Đệ quy: a * b = a + a + a + ... + a (b lần)
int mult_recur(int a, int b) {
    if (b == 1)
        return a;
    else
        return a + mult_recur(a, b - 1);
}
```

Bài toán nhân 2 số nguyên: a * b

C (C17 + GNU extensions)
known limitations

```c
// Đệ quy: a * b = a + a + a + ... + a (b lần)
int mult_recur(int a, int b) {
    if (b == 1)
        return a;
    else
        return a + mult_recur(a, b - 1);
}

int main() {
    int m = 5;
    int n = 3;
    int result = mult_recur(m, n);
    return 0;
}
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 15 of 17

Stack

main

m	int 5
n	int 3
result	int ?

mult_recur

a	int 5
b	int 3

mult_recur

a	int 5
b	int 2

Bài toán giai thừa (Factorial)

$$fac(n) = n \times fac\ (n - 1)$$

$$fac(1) = 1$$

- Trường hợp cơ sở?
  - Dừng gọi đệ quy (điều kiện dừng)

```c
// hàm tính giai thừa (đệ quy)
#include <stdio.h>
int factorial(int n) {
    if (n == 0) return 1;  // điều kiện dừng, base case
    return n * factorial(n - 1);  // gọi đệ quy
}

int main() {
    int n;
    printf("Nhap n: ");
    scanf("%d", &n);
    printf("Giai thua cua %d la: %d\n", n, factorial(n));
    return 0;
}
```

Bài toán lũy thừa (Exponentiation)

$$a^b = a \times a \times a \ldots \times a (b \text{ times})$$

$$= a \times (a \times a \ldots \times a \ (b - 1) \text{ times}))$$

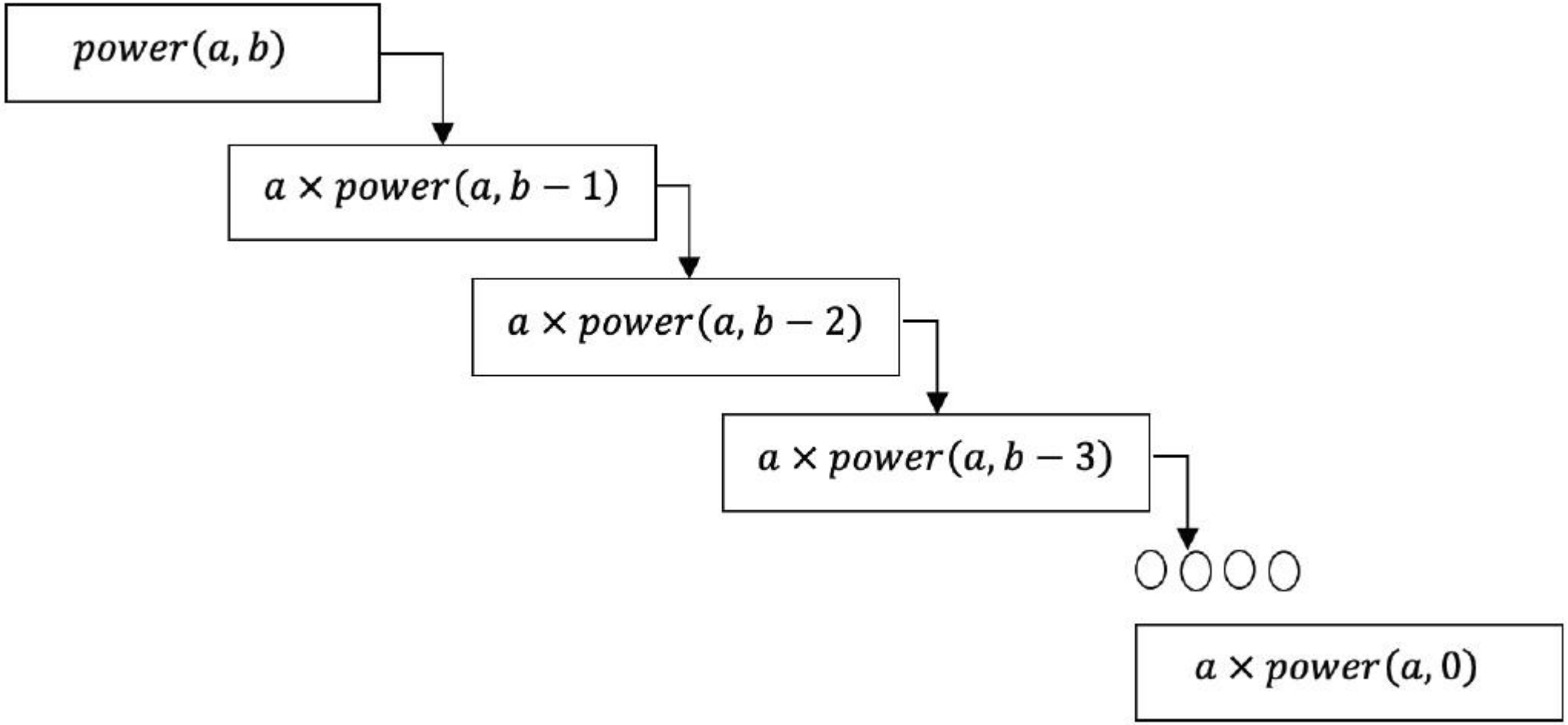
$$= a \times a^{b-1}$$

$$power(a,b) = a \times power\ (a, b\text{-}1)$$

- Quay về tính các lũy thừa nhỏ hơn: a^{b-1}
- Trường hợp cơ sở (base case): $a^0 = 1$

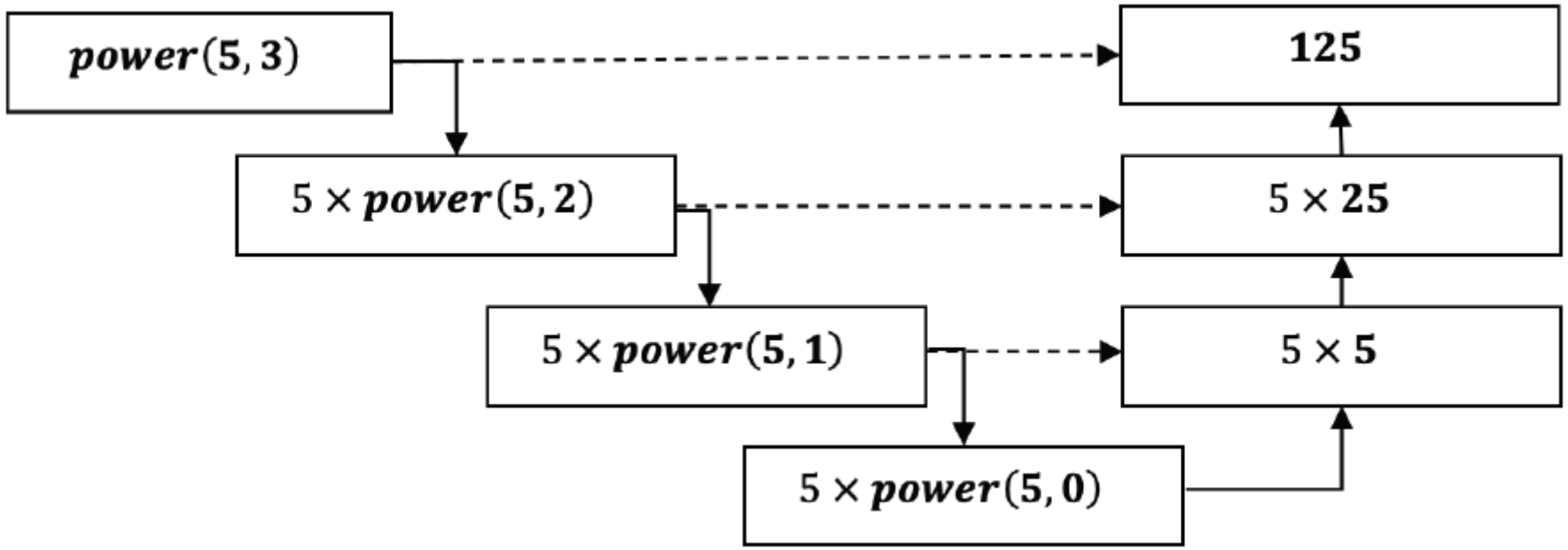
Bài toán lũy thừa (Exponentiation)

- Tính toán theo nguyên tắc Last In First Out (LIFO): a^0 tính đầu tiên



Bài toán lũy thừa (Exponentiation)

- Tính toán theo nguyên tắc Last In First Out (LIFO)



Bài toán lũy thừa (Exponentiation)

```c
// Đệ quy: Tính lũy thừa a^b (power.c)
#include <stdio.h>
int power(int a, int b) {
    if (b == 0) return 1;        // điều kiện dừng, base case
    return a * power(a, b - 1);  // gọi đệ quy
}

int main() {
    int a, b;
    printf("Nhap a va b: ");
    scanf("%d%d", &a, &b);
    printf("%d^%d = %d\n", a, b, power(a, b));
    return 0;
}
```

Đệ quy

- Nhận xét
  - Luôn luôn có thể chuyển một chương trình đệ quy thành chương trình không đệ quy (non-recursive program) để thực hiện cùng một công việc tính toán (same computation).

```
for (int t = 1, i = 1; i <= n; i++) {
    t = t * i;
}
```

  - Ngược lại, chúng ta có biểu diễn bất cứ tính toán nào liên quan đến vòng lặp (loops) mà không cần dùng vòng lặp bằng cách sử dụng đệ quy để khử.

Đệ quy

- Xét chương trình/hàm sau, ví dụ gọi puzzle(3) → kết quả?

```c
int puzzle(int n) {
    printf("puzzle(%d)\n", n);
    if (n == 1) return 1;
    if (n % 2 == 0)
        return puzzle(n / 2);
    else
        return puzzle(3 * n + 1);
}
```

```
puzzle(3)
  puzzle(10)
    puzzle(5)
      puzzle(16)
        puzzle(8)
          puzzle(4)
            puzzle(2)
              puzzle(1)
```

- Nhận xét?
  - Phải chỉ định trường hợp cơ sở (explicitly solve a basis case)
  - Mỗi lệnh gọi đệ quy phải bao gồm các giá trị nhỏ hơn của đối số (each recursive call must involve smaller values of the arguments).

Đệ quy: tìm ước số chung lớn nhất (GCD)

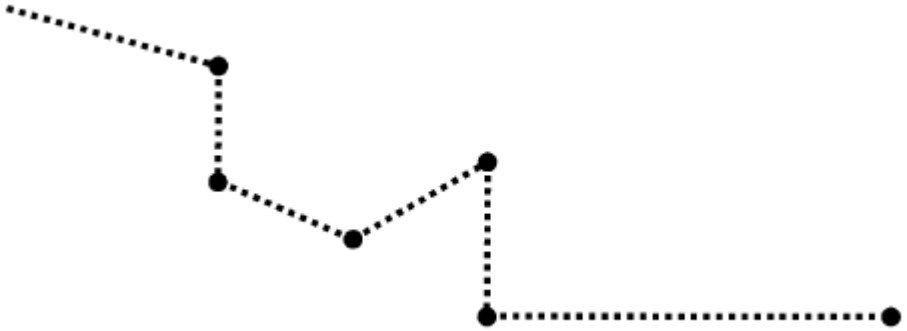
- Ước số chung (common divisor) của 2 số nguyên m, n:
  - Số chia hết cho 2 số nguyên m, n mà không để lại phần dư (remainder)
- Ước số chung lớn nhất (Greatest Common Divisor – GCD)
  - Số lớn nhất trong các ước số chung
- Tại sao `GCD(m, n) = GCD(n, m % n)`?

```
int gcd(int m, int n) {
    if (n == 0) return m;
    return gcd(n, m % n);
}
```

Phân loại đệ quy

- Đệ quy tuyến tính (Linear Recursion)
  - Trong thân hàm có duy nhất một lời gọi hàm gọi lại chính nó một cách tường minh
  - Ví dụ: bài toán tính giai thừa, lũy thừa

```
<Tên hàm>
{
    if<Điều kiện dừng>
    {
        // Trả về giá trị (Nếu có)
        // Kết thúc công việc
    }
    else
    {
        // Làm một số công việc (nếu có)
        // Gọi lại hàm <Tên hàm>
    }
}
```



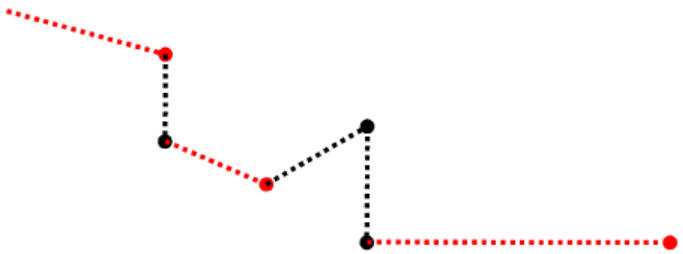
Phân loại đệ quy

- Đệ quy nhị phân/cây (Binary/Tree Recursion)
  - Trong thân hàm có hai lời gọi hàm gọi lại chính nó một cách tường minh (hay một hàm gọi đệ quy từ 2 lần trở lên → phân nhánh)
  - Bài toán tính số Fibonacci

```
<Tên hàm>
{
    if<Điều kiện dừng>
    {
        // Trả về giá trị (Nếu có)
        // Kết thúc công việc
    }
    else
    {
        // Làm một số công việc (nếu có)
        // Gọi lại hàm <Tên hàm> để giải quyết vấn đề thứ nhất

        // Gọi lại hàm <Tên hàm> để giải quyết vấn đề thứ hai
    }
}
```

Phân loại đệ quy

- Đệ quy hỗ tương (mutual recursion)
  - Trong thân hàm này có lời gọi hàm tới hàm kia và bên trong thân hàm kia có lời gọi hàm tới hàm này.



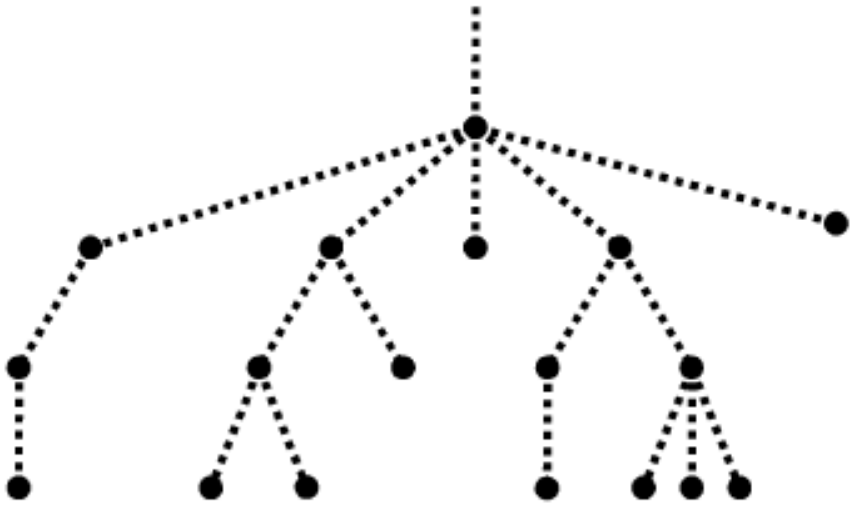
```
<Tên hàm 1>
{
    // Làm một số công việc (Nếu có)
    // Quá trình gọi Hàm 2
}

<Tên hàm 2>
{
    // Làm một số công việc (Nếu có)
    // Quá trình gọi Hàm 1
}
```

```
/*Cho 2 dãy số X{n} và Y{n} được định nghĩa như sau
X(0) = Y(0) = 1
X(n) = X(n-1) + Y(n-1);
Y(n) = n ^ 2 * X(n-1) + Y(n-1);
*/
int X(int n)
{
    if(n == 0)
        return 1;
    return X(n-1) + Y(n-1);
}

int Y(int n)
{
    if(n == 0)
        return 1;
    return n * n * X(n-1) + Y(n-1);
}
```

Phân loại đệ quy

- Đệ quy phi tuyến/đa tuyến (Multipe recursion)
  - Trong thân hàm có lời gọi hàm lại chính nó được đặt bên trong thân vòng lặp (gọi nhiều lần đệ quy bên trong hàm)
  - Bài toán tháp Hà Nội

```
<Tên hàm>
{
    if<Điều kiện dừng>
    {
        // Trả về giá trị (Nếu có)
        // Kết thúc công việc
    }
    else
    {
        // Lồng việc gọi đệ quy trong vòng lặp

        // Trả về giá trị
    }
}
```



Phân loại đệ quy

```
/* Ví dụ
Tính F(n):
+Với n = 0: F(n) = F(0) = 1
+Với n > 0: F(n) = 1^2 x F(1) + 2^2 x F(2) + ... + n^2 x F(n)
*/
int F(int n)
{
    if(n == 0)
        return 1;
    int f = 0;
    for(int i=1; i<= n; i++)
    {
        f+= i * i * F(n - 1 );
    }
    return f;
}
```

Phân loại đệ quy

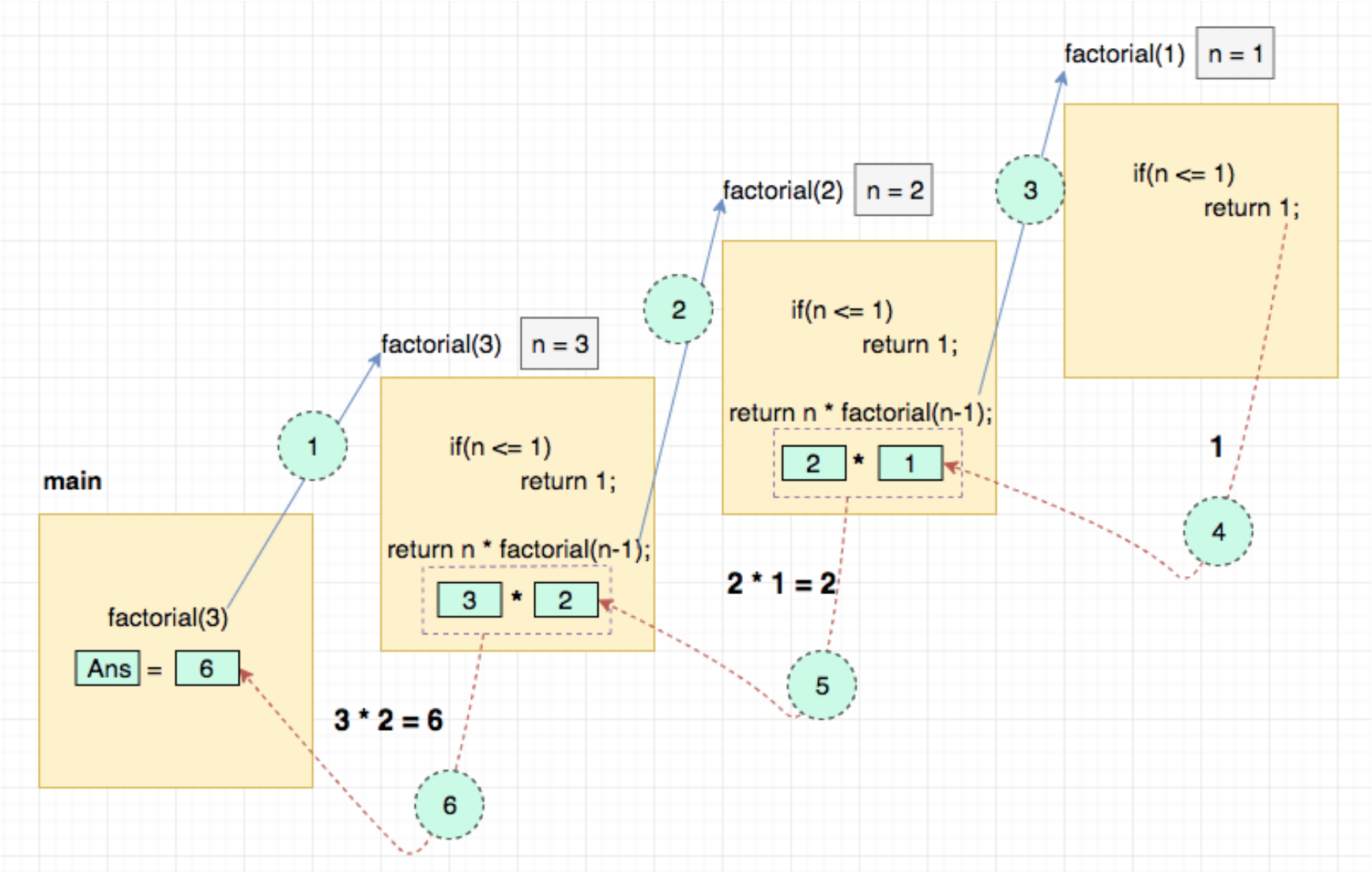
- Đệ quy phi tuyến/đa tuyến (Multipe recursion)
  - Trong thân hàm có lời gọi hàm lại chính nó được đặt bên trong thân vòng lặp (gọi nhiều lần đệ quy bên trong hàm)
  - Bài toán tháp Hà Nội
- Đệ quy đuôi (Tail recursion): lời gọi đệ quy cuối hàm

```
// tail recursion of sum
int recursive_sum(int n) {
    if (n == 0) return 0;
    return n + recursive_sum(n - 1);
}
```

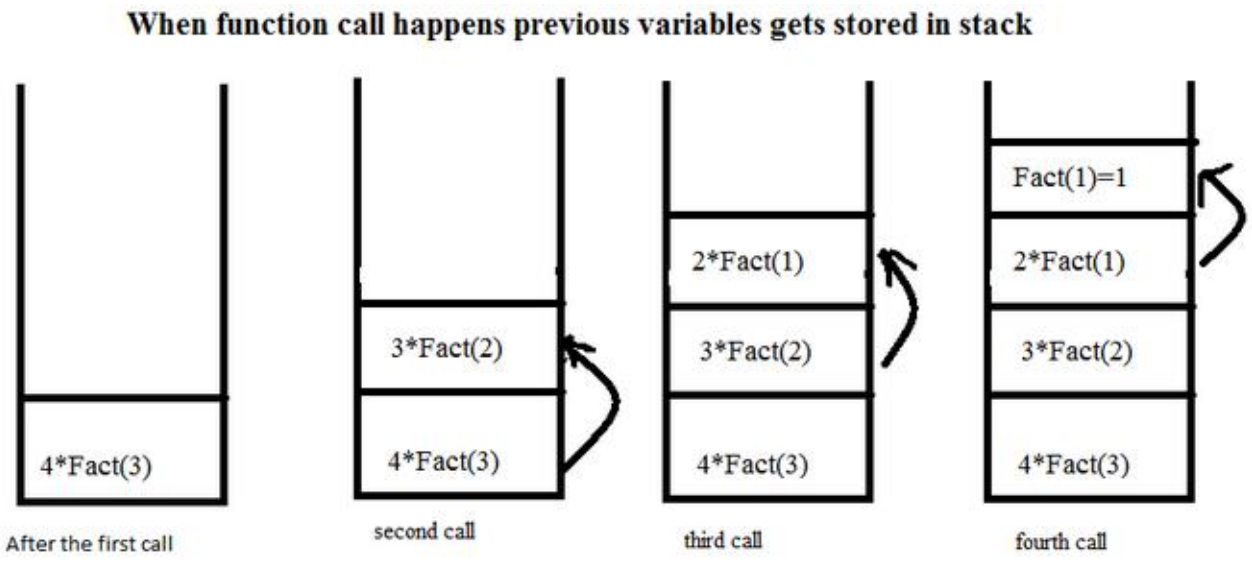
Ưu & khuyết điểm của đệ quy

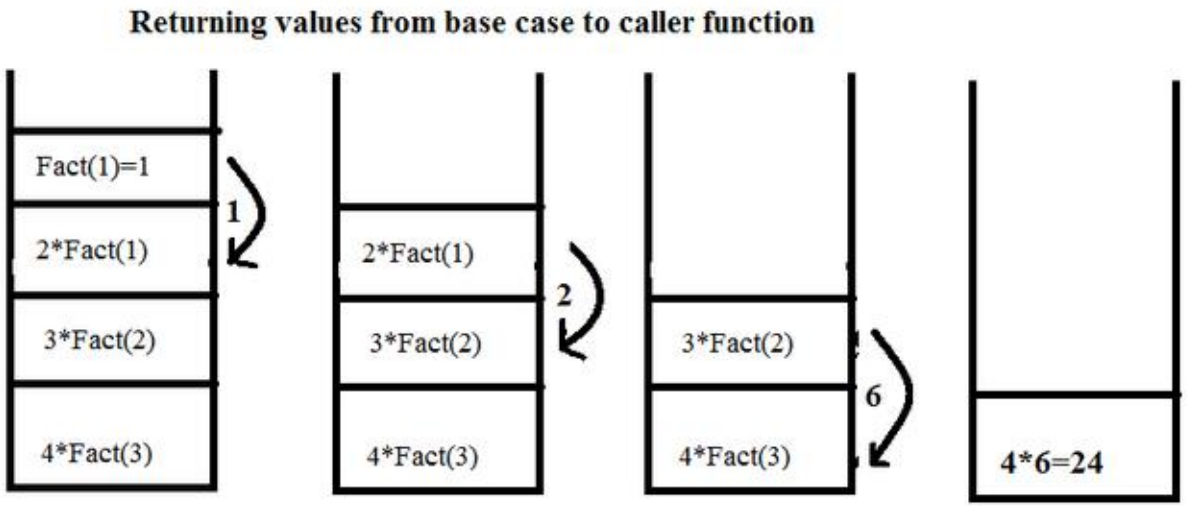
- Ưu điểm
  - Dễ dàng viết code
  - Giảm số lượng dòng code
  - Giải bài toán một cách tự nhiên
- Nhược điểm
  - Tốn nhiều bộ nhớ
  - Xử lý chậm hơn các hàm không đệ quy → không hiệu quả về không gian & thời gian (space & time complexity)
  - Khó phân tích code

Đệ quy và bộ nhớ stack



Đệ quy và bộ nhớ stack



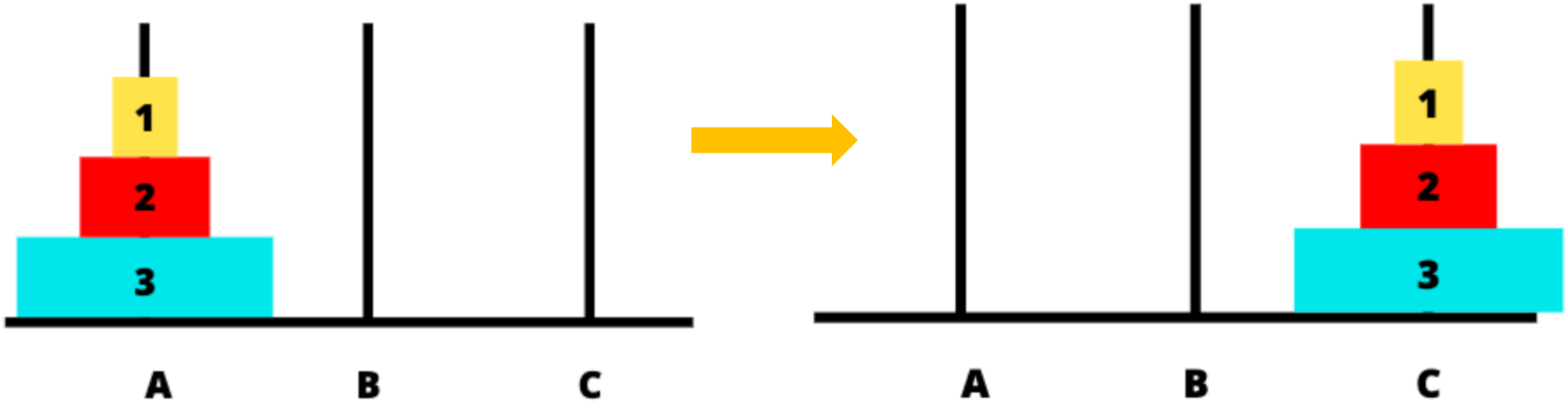


Khử đệ quy (removing recursion)

- Sử dụng vòng lặp (loop/iteration)
  - Xem lại các ví dụ tính lũy thừa, giai thừa, fibonacci...
- Sử dụng cấu trúc dữ liệu ngăn xếp (stack)
  - Sẽ trở lại trong bài Cấu trúc dữ liệu Stack

Bài toán tháp Hà Nội (Tower of Hanoi)

- Có 3 cọc (peg) theo thứ tự A, B, C và n đĩa xếp chồng nhau theo quy tắc bán kính giảm dần.
- Yêu cầu: di chuyển tất cả (n) đĩa từ cọc A sang cọc C và tuân thủ quy tắc:
  - Mỗi lần chỉ di chuyển 1 đĩa
  - Các đĩa tại mỗi cọc luôn được xếp theo thứ tự bán kính giảm dần

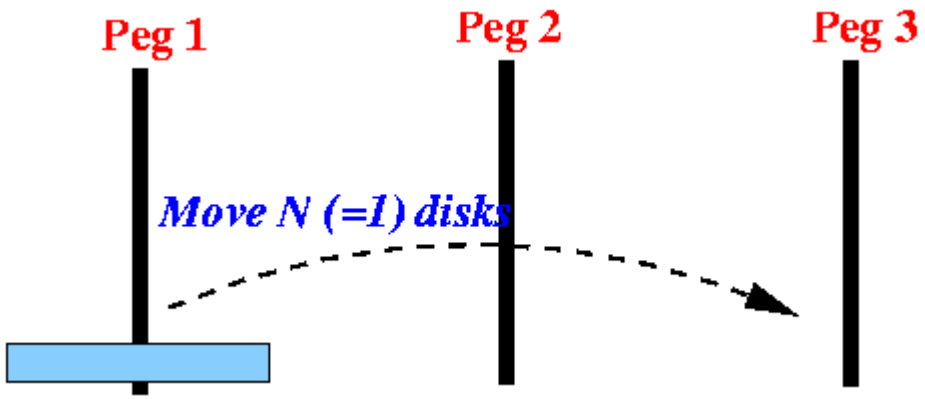


Bài toán tháp Hà Nội (Tower of Hanoi)

- Nhận xét, với n là số đĩa
  - n = 1: số lần di chuyển (moves) là 1
  - n = 2: số lần di chuyển là 3
  - n = 3: số lần di chuyển là 7
- Tổng quát
  - Với n đĩa, số lần di chuyển (moves) là **$m = (2^n - 1)$**
- Link
  - https://www.mathsisfun.com/games/towerofhanoi.html

Bài toán tháp Hà Nội (Tower of Hanoi)

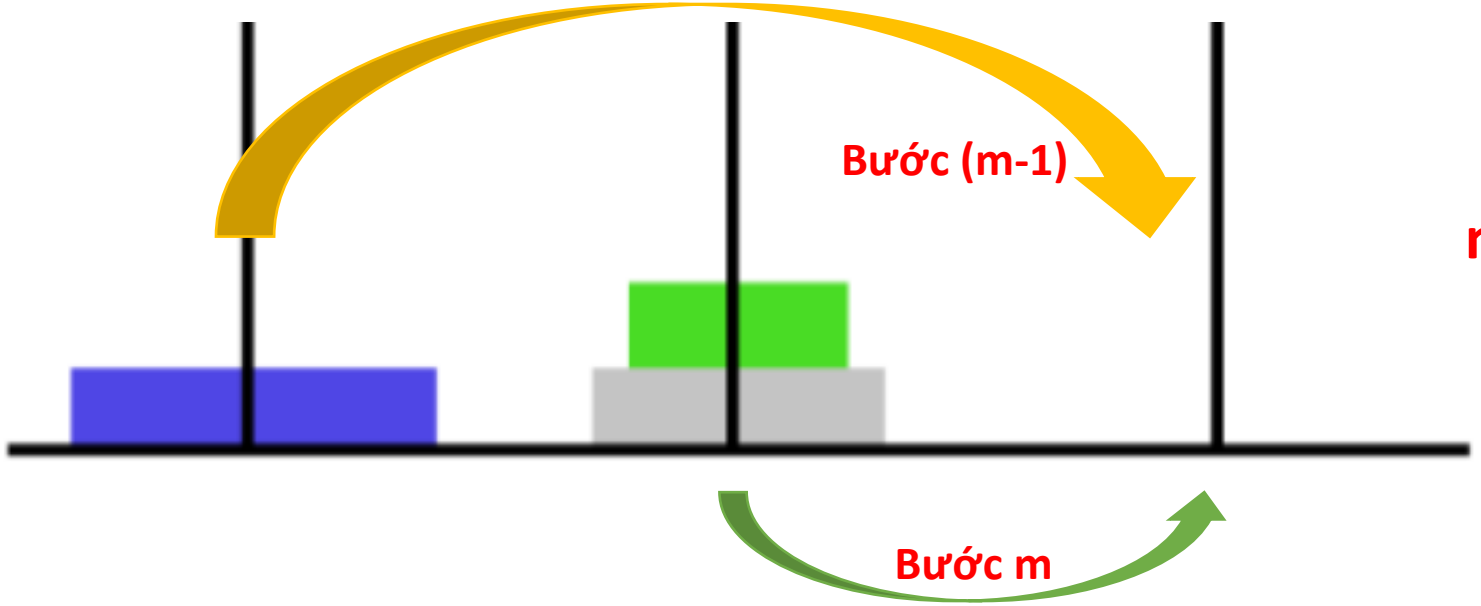
- Nhận xét tính chất đệ quy của bài toán?
  - Trường hợp cơ sở (n = 1)



  - Trường hợp n = 2: trạng thái cuối cùng trước khi đi đến trạng thái kết quả luôn là?

Bài toán tháp Hà Nội (Tower of Hanoi)

- Nhận xét tính chất đệ quy của bài toán?
  - Trường hợp n = 3: trạng thái cuối cùng trước khi đến trạng thái kết quả thì sao?

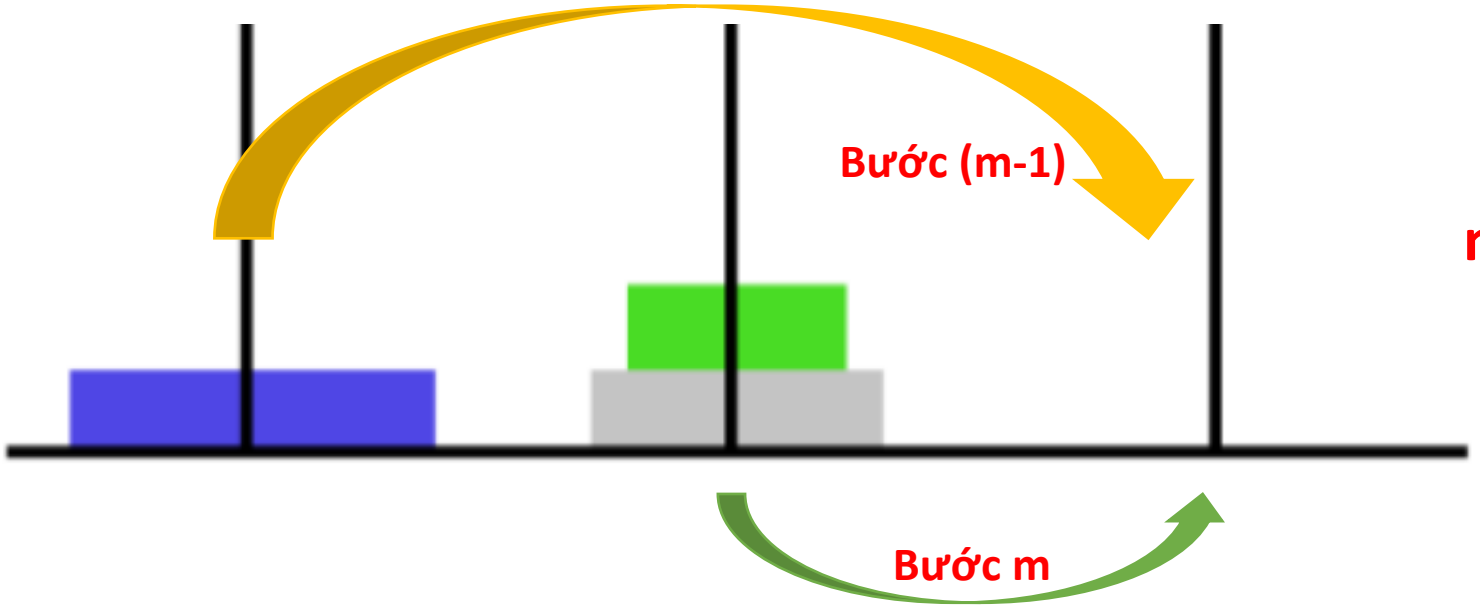


$m = (2^n - 1)$

  - Do đó, có thể đơn giản việc di chuyển n đĩa thành việc di chuyển 2 đĩa (tưởng tượng) gồm một đĩa lớn nhất (gọi là đĩa thứ n) và đĩa còn lại (chồng đĩa được tạo thành từ các đĩa 1 đến đĩa (n-1).

Bài toán tháp Hà Nội (Tower of Hanoi)

- Do đó, có thể tưởng tượng việc di chuyển n đĩa thành việc di chuyển 2 đĩa gồm một đĩa lớn nhất (gọi là đĩa thứ n) và đĩa còn lại (tạo thành từ chồng các đĩa 1 đến đĩa (n-1). Để làm được việc trên cần:
  - Di chuyển chồng đĩa (gồm đĩa 1 đến đĩa n-1) qua cọc phụ (auxiliary peg)
  - Di chuyển đĩa lớn nhất (đĩa n) qua cọc đích (destination peg)

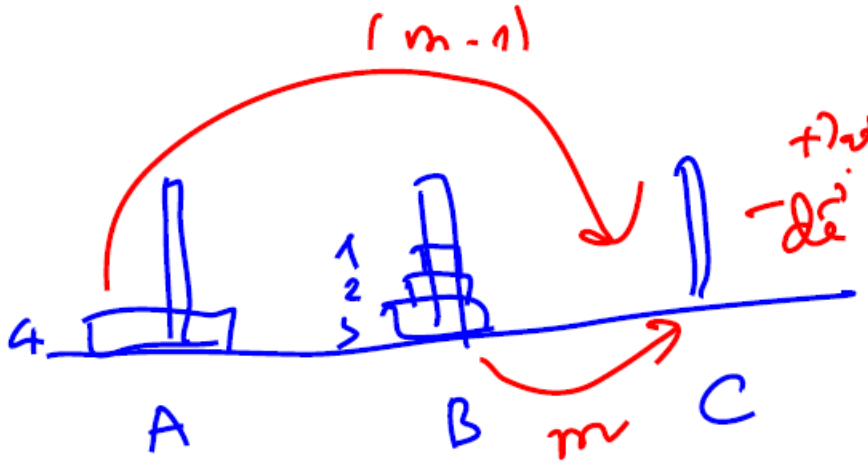


$m = (2^n - 1)$

Bài toán tháp Hà Nội (Tower of Hanoi)

Trường hợp: xem chồng n đĩa (1, 2, 3 ... n) gồm 2 đĩa lớn n và cái đĩa còn lại là 1 chồng compound (1, ... n-1)

=> thực hiện bài toán cho 2 đĩa



+) Đặt trong thời gian để disk 4 về đúng vị trí

Bài toán tháp Hà Nội (Tower of Hanoi)

```c
#include <stdio.h>

// Bài toán tháp Hà Nội (Tower of Hanoi)
// Thuật toán đệ quy (Recursion)

// num (number of disk): số lượng đĩa
// src (source peg): cọc ban đầu
// dest (destination peg): cọc đích các đĩa cần di chuyển đến
// aux (auxiliary peg): cọc phụ/trung gian
```

```c
int main() {
    // Lưu ý: C là cọc đích, theo thứ tự cọc A | cọc B | cọc C
    printf("+--------------------+\n");
    printf("| DISK | FROM | TO   |\n");
    printf("+--------------------+\n");
    tower_of_hanoi(3, 'A', 'C', 'B');
    printf("+--------------------+\n");
    return 0;
}
```

Bài toán tháp Hà Nội (Tower of Hanoi)

```c
void tower_of_hanoi(int num, char src, char dest, char aux) {
    // trường hợp cơ sở
    if (num == 1) {
        printf("|  %d   |  %c   |  %c   |\n", num, src, dest);
        return;
    } else {
        // Di chuyển chồng đĩa con (1 đến n-1) từ cọc src qua cọc aux.
        // dùng dest làm cọc trung gian
        tower_of_hanoi(num - 1, src, aux, dest);
        // Di chuyển đĩa lớn nhất (đĩa n) từ cọc src qua cọc dest
        printf("|  %d   |  %c   |  %c   |\n", num, src, dest);
        // Di chuyển chồng đĩa con từ cọc aux qua cọc dest
        // dùng src làm cọc trung gian
        tower_of_hanoi(num - 1, aux, dest, src);
    }
}
```

Bài toán tháp Hà Nội (Tower of Hanoi)

```
Move disk 1 from peg A to peg C
Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 3 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C
```

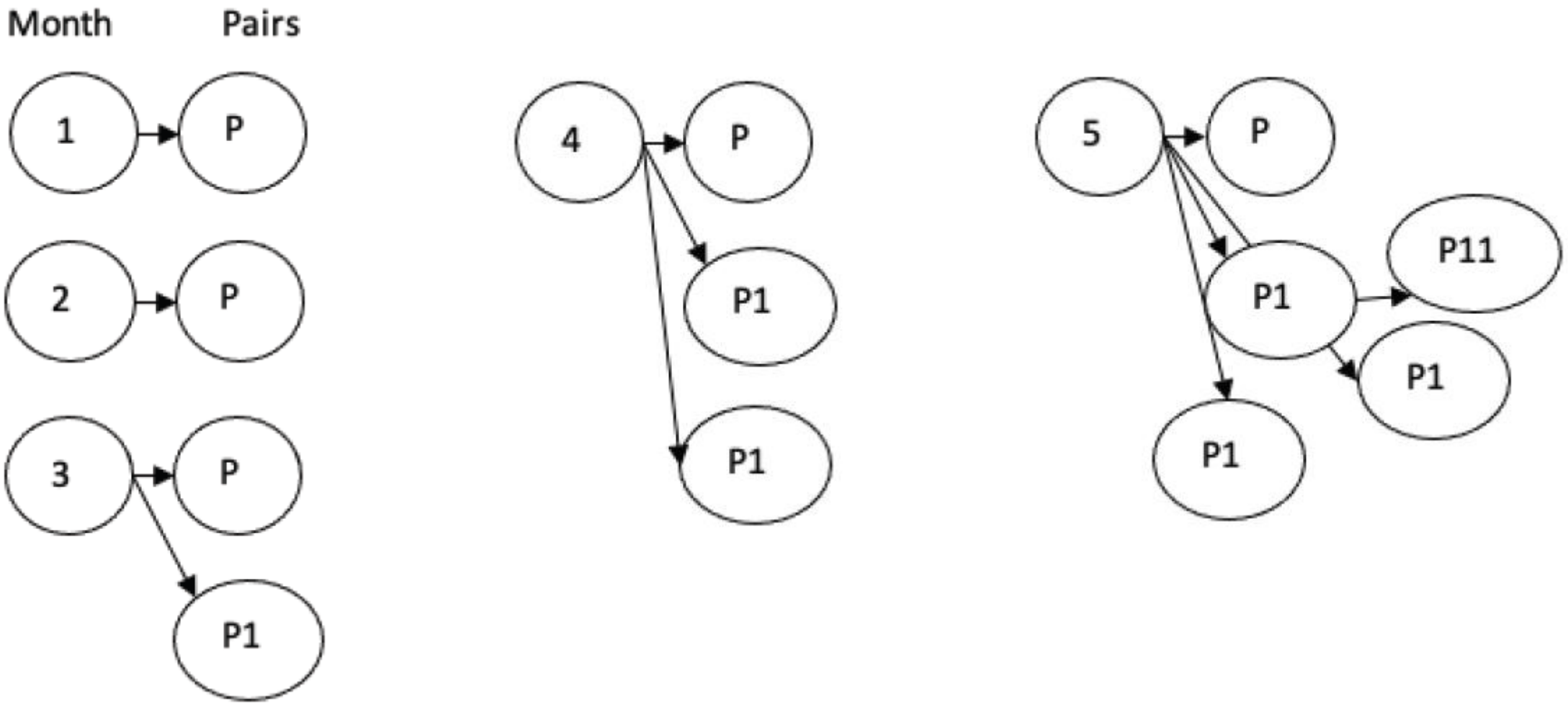
DISK	FROM	TO
1	A	C
2	A	B
1	C	B
3	A	C
1	B	A
2	B	C
1	A	C

Bài toán thỏ sinh sản (Rabbit problem)

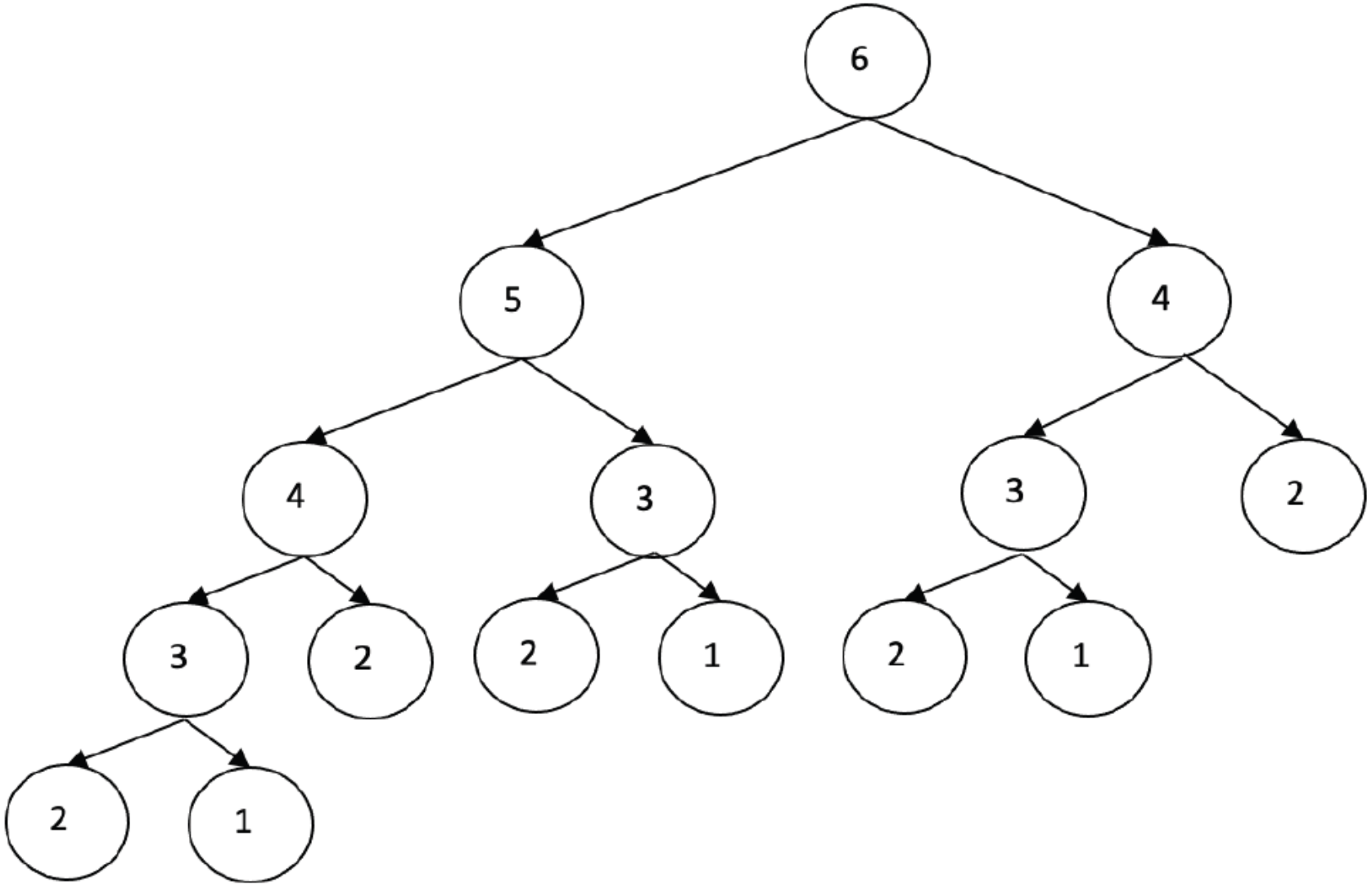
- Một cặp thỏ con, sau 2 tháng sẽ bắt đầu sinh sản
  - Mỗi lần sinh một cặp thỏ con
  - Sinh cách đều mỗi tháng
- Các cặp thỏ con lại tiếp tục quy luật trên...
- Vậy sau n tháng có bao nhiêu cặp con thỏ?
  - N = 1: 1 cặp
  - N = 2: 1 cặp
  - N = 3: 2 cặp
  - N = 4: 3 cặp
  - N = 5: 5 cặp
  - ...dãy Fibonacci

$$f(n) = \begin{cases} 1, n = 1 \\ 1, n = 2 \\ f(n-1) + f(n-2) \end{cases}$$

Bài toán thỏ sinh sản (Rabbit problem)



Dãy Fibonacci



Dãy Fibonacci

```c
#include <stdio.h>
#include <time.h>

// Tính phần tử thứ n trong dãy Fibonacci: fibonacci.c
// Sử dụng đệ quy (Recursion)
int fibonacci(int n) {
    if (n == 1 || n == 2)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

```

Dãy Fibonacci

```c
int main() {
    int n;
    printf("Enter n (0 < n < 40): ");
    scanf("%d", &n);

    // Thời gian thực thi
    clock_t start = clock();
    int f = fibonacci(n);
    clock_t end = clock();
    double elapsed = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Fibonacci(%d): %d\n", n, f);
    printf("Elapsed time: %.3f sec\n", elapsed);

    return 0;
}
```

```
Enter n (0 < n < 40): 30
Fibonacci(30): 832040
Elapsed time: 0.004 sec
```

Dãy Fibonacci

```c
#include <stdio.h>
#include <time.h>

// Tính phần tử thứ n trong dãy Fibonacci (fibonacci_2.c)
// Sử dụng Quy hoạch động (Dynamic programming)
// Không sử dụng đệ quy/Khử đệ quy (Removing recursion)
int fibonacci(int n) {
    // Sử dụng mảng để lưu kết quả tính toán của các phần tử trước
    // 2 phần tử đầu tiên là 1, 1
    int L[500] = {1, 1};
    int i;
    for (i = 2; i < n; i++) {
        L[i] = L[i - 1] + L[i - 2];
    }
    return L[n - 1];
}

```

Dãy Fibonacci

```c
int main() {
    int n;
    printf("Enter n (0 < n < 500): ");
    scanf("%d", &n);

    // Thời gian thực thi
    clock_t start = clock();
    long f = fibonacci(n);
    clock_t end = clock();
    double elapsed = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Fibonacci(%d): %d\n", n, f);
    printf("Elapsed time: %.3f sec\n", elapsed);

    return 0;
}
```

```
Enter n (0 < n < 500): 30
Fibonacci(30): 832040
Elapsed time: 0.000 sec
```

Độ phức tạp [thời gian] khi dùng đệ quy?

- Lưu ý
  - F(n) = F(n-1) + F(n-2)
- Giả sử độ phức tạp (Complexity) khi tính F(n) là *O(a^n)*
- Do đó
  - $a^n = a^{n-1} + a^{n-2}$
  - $chia\ 2\ vế\ cho\ a^{n-2}\ ta\ có:\ a^2 = a + 1$
  - $giải\ phương\ trình\ bậc\ hai\ a^2 - a - 1 = 0\ (a\ > 0)$
  - $\text{nghiệm}\ a = (1 + \sqrt{5})/2 \approx 1.618$
- Độ phức tạp [thời gian] khi dùng đệ quy để tính Fibonacci(n) là

$$O(1.618^n)$$

- Tỷ lệ/tỷ số vàng (*golden ratio*)

$$a = (1 + \sqrt{5})/2 \approx 1.618$$

Tỷ lệ vàng (golden ratio)

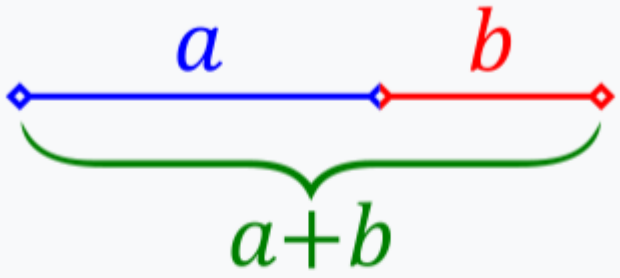
$$\frac{a+b}{a} = \frac{a}{b} = \varphi$$

$$1 + b/a = a/b = \varphi$$

$$1 + 1/\varphi = \varphi \leftrightarrow \varphi^2 - \varphi - 1 = 0$$

$$\varphi = \frac{1+\sqrt{5}}{2} = 1.618\ 033\ 988\ 749....$$

Golden ratio (φ)



$a+b$ is to a as a is to b

Representations

Decimal	1.618 033 988 749 894...[1]
Algebraic form	$\frac{1+\sqrt{5}}{2}$
Continued fraction	$1+\cfrac{1}{1+\cfrac{1}{1+\cfrac{1}{1+\ddots}}}$

Tìm phần tử lớn nhất trong danh sách

```c
#include <stdio.h>
// Tìm phần tử lớn nhất trong danh sách (find_max.c)
// Sử dụng đệ quy
int find_max(int a[], int n) {
    // base case: trường hợp mảng chỉ có 1 phần tử
    if (n == 1) {
        return a[0];
    } else {
        // recursive case: tìm phần tử lớn nhất trong mảng a[0..n-2]
        // so sánh phần tử lớn nhất trong mảng a[0..n-2] với phần tử cuối cùng
        // a[n-1]
        int max = find_max(a, n - 1);
        if (a[n - 1] > max)
            return a[n - 1];
        else
            return max;
        // có thể viết gọn như sau: return a[n - 1] > max ? a[n - 1] : max;
    }
}
```

Phát sinh các số nhị phân (Generating binary numbers)

- Xét tập hợp các số nhị phân gồm n-bit
  - n = 1 : {0, 1}
  - n = 2: {00, 01, 10, 11}
  - n = 3: {000, 001, 010, 011, 100, 101, 110, 111}
- Nhận xét, tập các số nhị phân n-bit được tạo thành bằng cách:
  - Thêm số 0 vào trước (hoặc sau) tập các số (n - 1) bit
  - Thêm số 1 vào trước (hoặc sau) tập các số (n - 1) bit
- Tính chất đệ quy
  - Base case: n = 1 gồm {0, 1}
  - {n bit} = {0 {n - 1 bit}, 1 {n - 1 bit}}
- Có thể phát sinh tập các số bất kỳ: bát phân (octal), thập phân (decimal), thập lục phân (hexa-decimal)...

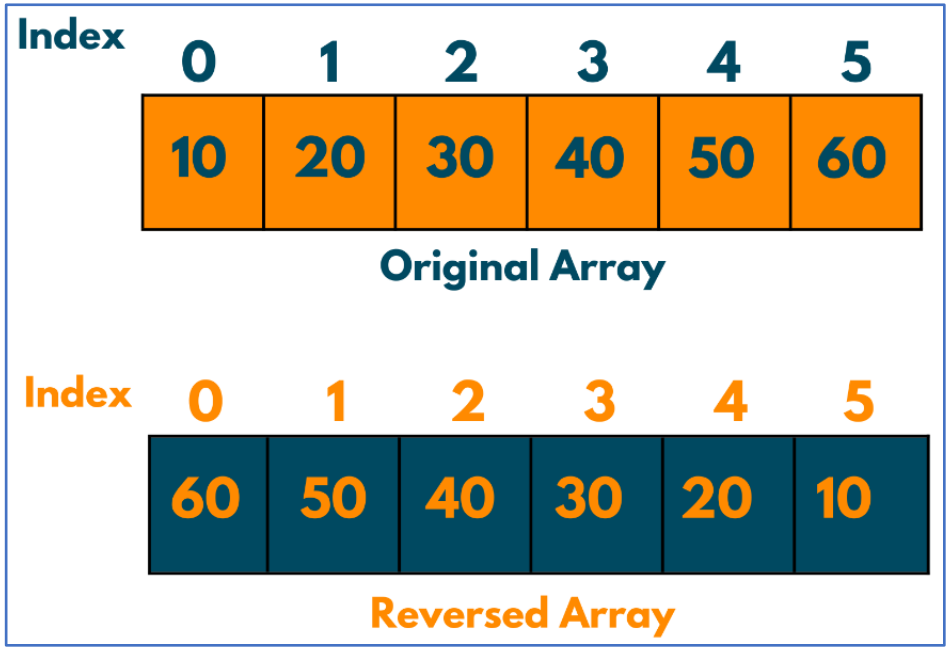
Phát sinh các số nhị phân (Generating binary numbers)

```c
// Đệ quy: phát sinh các số nhị phân n bits (generate_bin.c)
#include <stdio.h>
// i: thứ tự bit cần phát sinh
// a: mảng tạm chứa bit đã phát sinh
void generate_bin(int n, char *a, int i) {
    if (i == n) {
        a[i] = '\0';
        printf("%s\n", a);
        return;
    }
    a[i] = '0';
    generate_bin(n, a, i + 1);
    a[i] = '1';
    generate_bin(n, a, i + 1);
}

void main() {
    int n = 3;
    char a[n];
    generate_bin(n, a, 0);
}
```

```
000
001
010
011
100
101
110
111
```

Nghịch đảo các phần tử trong list (Reversed list)

- Cho một danh sách gồm n phần tử, nghịch đảo thứ tự các phần tử
- Nhận xét
  - n = 1: nghịch đảo là chính nó
  - n = 2: hoán vị phần tử cuối (n) lên đầu
  - n = 3: hoán vị phần tử cuối (n) lên đầu so với dãy (n-1) phần tử còn lại. Sau đó, thực hiện tương tự (đệ quy) với dãy (n-1) phần tử.
- Đệ quy
  - Base case: chỉ có 1 phần tử
  - [n phần tử] = [n, reverse(n-1)]
- Tìm phần tử lớn nhất trong danh sách?
  - Tương tự bài toán nghịch đảo



Nghịch đảo các phần tử trong list (Reversed list)

L_0	10	20	30	40	50	60
L_1	60	10	20	30	40	50
L_2	60	50	10	20	30	40
L_3	60	50	40	10	20	30
L_4	60	50	40	30	10	20
L_5	60	50	40	30	20	10

Nghịch đảo các phần tử trong list (Reversed list)

```c
#include <stdio.h>
// Nghịch đảo thứ tự các phần tử trong danh sách
// Reversed list
void reverse(int L[], int start, int end) {
    // Base case
    if (start >= end) {
        return;
    } else {
        // Hoán vị 2 phần tử đầu và cuối
        int temp = L[start];
        L[start] = L[end];
        L[end] = temp;
        // Đệ quy, tiếp tục nghịch đảo các phần tử còn lại
        reverse(L, start + 1, end - 1);
    }
}

```

Nghịch đảo các phần tử trong list (Reversed list)

```c
int main() {
    int L[] = {10, 20, 30, 40, 50, 60};
    int n = sizeof(L) / sizeof(L[0]);

    reverse(L, 0, n - 1);

    // Print the reversed list
    int i;
    for (i = 0; i < n; i++) {
        printf("%d ", L[i]);
    }

    return 0;
}
```

```
60 50 40 30 20 10
```

Nghịch đảo các chữ số (Reversed numbers)

- Cho một số tự nhiên gồm n chữ số, nghịch đảo thứ tự các chữ số.
- Nhận xét:
  - n = 1 (số chỉ có 1 chữ số): nghịch đảo là chính nó
  - n = 2: lấy giá trị chữ số cuối cùng nhân 10^1 + với giá trị của chữ số đầu tiên
  - n = 3: lấy giá trị chữ số cuối cùng nhân 10^2 + nghịch đảo của 2 số đầu tiên...
- Đệ quy
  - Base case: số chỉ có 1 chữ số

Nghịch đảo các chữ số (Reversed numbers)

```c
// Nghịch đảo các chữ số (Reverse numbers)
// (reverse_num.c)
// Ví dụ: 12345 --> 54321
#include <stdio.h>

void reverse_num(int n) {
    if (n == 0) {
        return;
    } else {
        printf("%d", n % 10);
        reverse_num(n / 10);
    }
}

int main() {
    int n = 12345;
    reverse_num(n);
    return 0;
}
```

Nghịch đảo các chữ số (Reversed numbers)

```c
// Nghịch đảo các chữ số (Reverse numbers)
// (reverse_num_2.c)
// Ví dụ: 12345 --> 54321
#include <math.h>
#include <stdio.h>
// Đếm số lượng các chữ số của số nguyên dương n
int count_digits(int n) {
    if (n < 10) {
        return 1;
    } else {
        return 1 + count_digits(n / 10);
    }
}
```

Bài tập:
Khử đệ quy hàm count_digits()

Nghịch đảo các chữ số (Reversed numbers)

```c
int reverse_num(int n) {
    if (count_digits(n) == 1) {
        return n;
    } else {
        int last = n % 10;
        int first = n / 10;
        int d = count_digits(first);
        int rev_first = reverse_num(first);
        int temp = last * pow(10, d) + rev_first;
        return temp;
    }
}

int main() {
    int n = 12345;
    int m = reverse_num(n);
    printf("%d", m);
    return 0;
}
```